

Memoria Técnica: Funcionalidades Core en Android

Desarrollo Android

18 de marzo de 2026

Índice

1. Mapas con OSMDroid (OpenStreetMap)	2
2. Localización y GPS	2
3. Interacción por Voz (Text-To-Speech)	3
4. Sensores de Hardware (Acelerómetro)	3
5. Notificaciones en Barra de Estado	3
6. Programación de Notificaciones Diferidas (WorkManager)	4
7. Añadir Evento al Calendario Nativo	4
8. Navegador y Buscador Interno (WebView)	5
9. Extracción de Datos Web (Web Scraping con Jsoup)	5
10. Gráficas Complejas e Intuitivas (MPAndroidChart)	6
11. Acceso a la Cámara (ActivityResult API)	6

1. Mapas con OSMDroid (OpenStreetMap)

Notas explicativas: OSMDroid permite usar mapas libres y colaborativos sin depender de los servicios de Google. Requiere configurar permisos de red y establecer un "User-Agent" para identificarse ante el servidor y evitar bloqueos.

```
1 <uses-permission android:name="android.permission.INTERNET"/>
2 <uses-permission android:name="android.permission.ACCESS_NETWORK_STATE"/>
```

Listing 1: Permisos en AndroidManifest.xml

```
1 // Dependencia Gradle: implementation("org.osmdroid:osmdroid-android
   :6.1.18")
2
3 private lateinit var map: MapView
4 private var marcador: Marker? = null
5
6 // En onCreate()
7 Configuration.getInstance().userAgentValue = packageName
8 map = findViewById(R.id.map)
9 map.setMultiTouchControls(true)
10 map.controller.setZoom(15.0)
11
12 // Actualizar mapa (onLocationChanged)
13 val punto = GeoPoint(location.latitude, location.longitude)
14 map.controller.setCenter(punto)
15 marcador?.let { map.overlays.remove(it) }
16 marcador = Marker(map).apply {
17     position = punto
18     title = "Mi ubicaci n"
19 }
20 map.overlays.add(marcador)
21 map.invalidate()
```

Listing 2: Configuración de OSMDroid en Kotlin

2. Localización y GPS

Notas explicativas: Utiliza `LocationManager` para acceder a los servicios de ubicación. Requiere solicitar permisos en tiempo de ejecución para acceder de forma precisa.

```
1 private val permisoGPSLauncher = registerForActivityResult(
   ActivityResultContracts.RequestPermission()) { concedido ->
2     if (concedido) { iniciarGPS() }
3 }
4
5 // Iniciar GPS
6 locationManager = getSystemService(Context.LOCATION_SERVICE) as
   LocationManager
7 locationManager.requestLocationUpdates(LocationManager.GPS_PROVIDER,
   2000, 1f, this)
8
9 // Recibir nueva ubicaci n
10 override fun onLocationChanged(location: Location) {
11     val lat = location.latitude
12     val lon = location.longitude
```

```
13 }
```

Listing 3: Solicitud de actualizaciones de ubicación

3. Interacción por Voz (Text-To-Speech)

Notas explicativas: Permite simular un comportamiento cotidiano generando voz a partir de texto. Es vital liberar los recursos (`tts.shutdown()`) al cerrar la app para evitar fugas de memoria.

```
1 private lateinit var tts: TextToSpeech
2
3 tts = TextToSpeech(this) { status ->
4     if (status == TextToSpeech.SUCCESS) {
5         tts.language = Locale("es", "ES")
6     }
7 }
8
9 // Ejecutar mensaje
10 tts.speak(texto, TextToSpeech.QUEUE_FLUSH, null, null)
```

Listing 4: Inicialización y ejecución de TTS

4. Sensores de Hardware (Acelerómetro)

Notas explicativas: Gestionado mediante `SensorManager`. El acelerómetro devuelve los datos de aceleración (incluyendo la gravedad) en los ejes X, Y y Z.

```
1 sensorManager = getSystemService(Context.SENSOR_SERVICE) as
    SensorManager
2 sensorManager.getDefaultSensor(Sensor.TYPE_ACCELEROMETER)?.let {
3     accelerometer = it
4 }
5 sensorManager.registerListener(this, accelerometer, SensorManager.
    SENSOR_DELAY_NORMAL)
6
7 override fun onSensorChanged(event: SensorEvent?) {
8     event?.let {
9         val ejeX = it.values[0]
10        val ejeY = it.values[1]
11        val ejeZ = it.values[2]
12    }
13 }
```

Listing 5: Registro y lectura del acelerómetro

5. Notificaciones en Barra de Estado

Notas explicativas: En Android Oreo (API 26) o superior, es obligatorio crear un “Canal de notificaciones”. Se utiliza `PendingIntent` para acciones al tocar la notificación.

```
1 if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
2     val canal = NotificationChannel(CHANNEL_ID, "Alertas",
        NotificationManager.IMPORTANCE_HIGH)
```

```

3      getSystemService(NotificationManager::class.java).
        createNotificationChannel(canal)
4  }
5
6  val pendingIntent = PendingIntent.getActivity(this, 0, intent,
        PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE)
7
8  val builder = NotificationCompat.Builder(this, CHANNEL_ID)
9      .setSmallIcon(android.R.drawable.ic_dialog_alert)
10     .setContentTitle("Alerta")
11     .setContentText("Pulsa para m s informaci n")
12     .setContentIntent(pendingIntent)
13     .setAutoCancel(true)
14
15 NotificationManagerCompat.from(this).notify(1001, builder.build())

```

Listing 6: Creación de Canal y Lanzamiento de Notificación

6. Programación de Notificaciones Diferidas (WorkManager)

Notas explicativas: Para lanzar notificaciones pasado X tiempo o en una fecha concreta de manera fiable, se utiliza **WorkManager**. Este sobrevive a reinicios del dispositivo y optimiza el uso de la batería.

```

1 // Dependencia: implementation("androidx.work:work-runtime-ktx:2.8.1")
2
3 class NotificacionWorker(context: Context, params: WorkerParameters) :
    Worker(context, params) {
4     override fun doWork(): Result {
5         lanzarNotificacion() // Llama al m todo del Builder de la
        secci n 5
6         return Result.success()
7     }
8 }
9
10 // Programar para dentro de 15 minutos
11 val solicitud = OneTimeWorkRequestBuilder<NotificacionWorker>()
12     .setInitialDelay(15, TimeUnit.MINUTES)
13     .build()
14
15 WorkManager.getInstance(context).enqueue(solicitud)

```

Listing 7: Programar tarea con retraso (WorkManager)

7. Añadir Evento al Calendario Nativo

Notas explicativas: Se utiliza un Intent implícito con **CalendarContract** para abrir la aplicación de calendario del usuario con los datos pre-rellenados, sin requerir permisos especiales de escritura.

```

1 val intent = Intent(Intent.ACTION_INSERT).apply {
2     data = CalendarContract.Events.CONTENT_URI
3     putExtra(CalendarContract.Events.TITLE, "Reuni n de Proyecto")

```

```

4     putExtra(CalendarContract.Events.EVENT_LOCATION, "Bilbao")
5     putExtra(CalendarContract.EXTRA_EVENT_BEGIN_TIME, System.
currentMillis())
6     putExtra(CalendarContract.EXTRA_EVENT_END_TIME, System.
currentMillis() + 3600000)
7 }
8 startActivity(intent)

```

Listing 8: Intent para añadir evento al calendario

8. Navegador y Buscador Interno (WebView)

Notas explicativas: WebView permite renderizar contenido web dentro de la aplicación. Configurando un WebViewClient, evitamos que los enlaces abran navegadores externos. Esto sirve tanto para páginas normales como para mostrar resultados de motores de búsqueda pasando la query en la URL.

```

1 val webView: WebView = findViewById(R.id.miWebView)
2
3 // Habilitar JavaScript si es necesario
4 webView.settings.javaScriptEnabled = true
5 // Evitar que abra Chrome externamente
6 webView.webViewClient = WebViewClient()
7
8 // 1. Acceder a una web espec fica
9 webView.loadUrl("https://www.google.com")
10
11 // 2. Mostrar resultados de un buscador din micamente
12 val query = "desarrollo android"
13 val urlBuscador = "https://www.google.com/search?q=${Uri.encode(query)}"
14 webView.loadUrl(urlBuscador)

```

Listing 9: Configuración de WebView y Buscador

9. Extracción de Datos Web (Web Scraping con Jsoup)

Notas explicativas: Jsoup es una librería para analizar HTML. Al implicar peticiones de red, debe ejecutarse fuera del hilo principal (usando Corrutinas en Kotlin).

```

1 // Dependencia: implementation("org.jsoup:jsoup:1.15.3")
2
3 lifecycleScope.launch(Dispatchers.IO) {
4     try {
5         val document = Jsoup.connect("https://es.wikipedia.org/wiki/
Android").get()
6         // Extraer el primer p rrafo
7         val primerParrafo = document.select("p").first()?.text()
8
9         withContext(Dispatchers.Main) {
10             textView.text = primerParrafo
11         }
12     } catch (e: Exception) {
13         e.printStackTrace()
14     }
15 }

```

15 }

Listing 10: Scraping asíncrono con Jsoup

10. Gráficas Complejas e Intuitivas (MPAndroidChart)

Notas explicativas: MPAndroidChart es el estándar de la industria para crear gráficos interactivos (barras, líneas, tarta) en Android. Requiere configurar los ejes y poblar los datos mediante `Entries`.

```
1 // Repositorio en settings.gradle: maven { url 'https://jitpack.io' }
2 // Dependencia: implementation("com.github.PhilJay:MPAndroidChart:v3
  .1.0")
3
4 val barChart: BarChart = findViewById(R.id.barChart)
5 val entradas = ArrayList<BarEntry>()
6
7 entradas.add(BarEntry(1f, 10f)) // Eje X, Eje Y
8 entradas.add(BarEntry(2f, 25f))
9
10 val dataSet = BarDataSet(entradas, "Datos de Ejemplo")
11 dataSet.color = Color.BLUE
12
13 val data = BarData(dataSet)
14 barChart.data = data
15 barChart.invalidate() // Refrescar gráfica
```

Listing 11: Implementación de BarChart

11. Acceso a la Cámara (Activity Result API)

Notas explicativas: La forma más moderna y limpia de obtener una miniatura de la cámara sin requerir permisos de escritura complicados es usando los nuevos contratos `ActivityResultContracts`.

```
1 val imageView: ImageView = findViewById(R.id.miImagen)
2
3 val takePictureLauncher = registerForActivityResult(
4     ActivityResultContracts.TakePicturePreview()) { bitmap ->
5     if (bitmap != null) {
6         imageView.setImageBitmap(bitmap)
7     }
8 }
9
10 // Para abrir la cámara
11 takePictureLauncher.launch(null)
```

Listing 12: Capturar foto y mostrar en ImageView